

Documentación de la Red Neuronal para Evaluación Automática de Prácticas

Esta documentación describe la arquitectura, entrenamiento, evaluación y uso del sistema de evaluación automática de prácticas educativas, basado en una red neuronal profunda.

Índice

1. Arquitectura de la Red Neuronal
2. Proceso de Entrenamiento
3. Metodología de Evaluación
4. Preprocesamiento de Datos
5. Generación de Retroalimentación
6. Generación de Prácticas
7. Uso del Sistema
8. Consideraciones y Limitaciones

1. Arquitectura de la Red Neuronal

El modelo central, denominado **PracticaEvaluator**, es una red neuronal profunda *feed-forward*, diseñada para asignar calificaciones automáticas a prácticas escritas en formato libre.

Estructura de Capas

```
class PracticaEvaluator(nn.Module):
    def __init__(self, input_dim=5000, hidden_dim=512, output_dim=1):
        super(PracticaEvaluator, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.dropout1 = nn.Dropout(0.3)

        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.dropout2 = nn.Dropout(0.3)

        self.fc3 = nn.Linear(hidden_dim, hidden_dim // 2)
        self.bn3 = nn.BatchNorm1d(hidden_dim // 2)
        self.dropout3 = nn.Dropout(0.3)

        self.fc4 = nn.Linear(hidden_dim // 2, hidden_dim // 4)
        self.bn4 = nn.BatchNorm1d(hidden_dim // 4)
        self.dropout4 = nn.Dropout(0.2)

        self.fc5 = nn.Linear(hidden_dim // 4, output_dim)

        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid() # Para salida entre 0 y 1
```

Características Técnicas

Atributo	Valor
Dimensión de entrada	5000 (vectores TF-IDF)
Capas ocultas	4 (con reducción progresiva de tamaño)
Activaciones	ReLU en capas ocultas, Sigmoid en salida
Batch Normalization	Presente en cada capa oculta

Regularización	Dropout de 0.2–0.3
Salida	Valor escalar en rango [0, 10]

Flujo de Datos en el Forward Pass

Diagrama (en texto):

Entrada (5000)
 ↓
 FC1 (512) → BatchNorm → ReLU → Dropout(0.3)
 ↓
 FC2 (512) → BatchNorm → ReLU → Dropout(0.3)
 ↓
 FC3 (256) → BatchNorm → ReLU → Dropout(0.3)
 ↓
 FC4 (128) → BatchNorm → ReLU → Dropout(0.2)
 ↓
 FC5 (1) → Sigmoid → Escalado (*10)
 ↓
 Salida (calificación 0-10)

Plantillas de Retroalimentación

El sistema utiliza una variedad de plantillas según el nivel de desempeño del estudiante, lo cual permite generar retroalimentaciones personalizadas y pertinentes.

Plantillas por nivel de desempeño:

- **Excelente:**
 - "Excelente trabajo. Has demostrado un dominio completo del tema, con un análisis profundo y bien estructurado."
 - "Trabajo sobresaliente. Cumples con todos los requisitos y vas más allá, demostrando comprensión avanzada de los conceptos."
 - *(Más plantillas disponibles...)*
- **Muy bueno:**
 - "Muy buen trabajo. Demuestras una sólida comprensión del tema con un análisis bien desarrollado."

- (Más plantillas disponibles...)
- (Se incluyen más categorías similares como: Bueno, Aceptable, Insuficiente...)

Generación de Prácticas

El sistema puede generar nuevas prácticas educativas a partir de un título y un objetivo de aprendizaje.

Proceso de Generación (Método `generar_practica`)

```
def generar_practica(self, titulo, objetivo):
    """
    Genera una práctica completa y detallada basada en el título y objetivo.
    """
    prompt = f"{titulo}. {objetivo}"
    prompt_vec = self.vectorizer.transform([prompt])
    similarities = cosine_similarity(prompt_vec, self.tfidf_matrix)
    top_indices = similarities[0].argsort()[-3:][::-1]
    practicas_similares = [self.base_conocimiento[i] for i in top_indices]
    categoria = self._determinar_categoria_tematica(titulo, objetivo)

    nueva_practica = {
        "titulo": titulo,
        "objetivo": objetivo,
        "descripcion": descripcion,
        "categoria": categoria,
        "instrucciones": instrucciones,
        "recursos": recursos,
        "criterios_evaluacion": criterios_evaluacion,
        "objetivos_aprendizaje": objetivos_aprendizaje,
        "tiempo_estimado": "2-3 horas",
        "nivel_dificultad": "Intermedio"
    }

    return nueva_practica
```

Base de Conocimiento

Se utiliza una base de conocimiento especializada para generar prácticas relevantes.

```
self.base_conocimiento = [
    "Resolver ecuaciones diferenciales de primer orden utilizando métodos numéricos.",
```

```

"Análisis de datos utilizando técnicas estadísticas descriptivas e inferenciales.",
"Implementación de algoritmos de ordenamiento y búsqueda para estructuras de datos.",
# ... más ejemplos
]

```

Evaluación de Contenido

El sistema evalúa automáticamente entregas de los estudiantes, generando una calificación y retroalimentación detallada.

Método **analizar_contenido**

```

def analizar_contenido(self, contenido_archivo, titulo_actividad="", objetivo_actividad="",
estilo_aprendizaje=None):
    """
    Analiza el contenido del archivo y devuelve una calificación y comentarios detallados.
    Incorpora análisis semántico profundo, evaluación multidimensional y personalización por
    estilo de aprendizaje.
    """
    # Validación y análisis...
    return {
        "calificacion": round(calificacion_final, 1),
        "comentarios": comentarios,
        "sugerencias": sugerencias,
        "relevancia": round(relevancia * 100, 1),
        "metricas": {
            "claridad": round(metricas['claridad'] * 10, 1),
            "profundidad": round(metricas['profundidad'] * 10, 1),
            "estructura": round(metricas['estructura'] * 10, 1),
            "originalidad": round(metricas['originalidad'] * 10, 1)
        },
        "fortalezas": fortalezas[:3],
        "debilidades": debilidades[:3],
        "recursos_recomendados": recursos_recomendados
    }

```

Entrenamiento del Modelo# Recopilar y dividir los datos

```

entradas, calificaciones = recopilar_datos_entrenamiento()
X_train, X_val, y_train, y_val = train_test_split(entradas, calificaciones, test_size=0.2,
random_state=42)

```

Entrenamiento

```
generador_ml.entrenar_modelo(X_train, y_train, epochs=50, batch_size=8)
```

Evaluación y guardado

```
evaluacion = evaluar_modelo(X_val, y_val)
```

```
print(f"Evaluación del modelo: {evaluacion}")
```

```
torch.save(generador_ml.model.state_dict(), "modelo_practicas.pth")
```

 Ejemplo de Usogenerador_ml = GeneradorPracticasML()

```
nueva_practica = generador_ml.generar_practica(
```

```
    titulo="Análisis de algoritmos de ordenamiento",
```

```
    objetivo="Comparar la eficiencia de diferentes algoritmos de ordenamiento mediante  
análisis teórico y empírico"
```

```
)
```

```
generador_db.guardar_contenido_generado(practica_id, nueva_practica)
```


2. Proceso de Entrenamiento

El entrenamiento del modelo implementa técnicas avanzadas para optimizar el rendimiento y prevenir el sobreajuste.

Configuración del Entrenamiento

- Optimizador: Adam con `weight_decay=1e-5`
- Función de Pérdida: Error Cuadrático Medio (MSE)
- Learning Rate: Inicial de `0.001`, ajustado dinámicamente con scheduling
- Batch Size: Por defecto `8`
- Epochs: Por defecto `50`

Técnicas de Optimización

1. Early Stopping

Detiene el entrenamiento si la pérdida de validación deja de mejorar.

python

```
if patience_counter >= patience:
    logger.info(f'Early stopping en epoch {epoch+1}')
    break
```

2. Learning Rate Scheduling

Reduce la tasa de aprendizaje si la pérdida se estanca.

python

```
scheduler = optim.lr_scheduler.ReduceLR0nPlateau(
    optimizer, mode='min', factor=0.5, patience=3, verbose=True
)
```

3. Gradient Clipping

Previene explosiones de gradiente.

python

```
torch.nn.utils.clip_grad_norm_(self.model.parameters(),  
max_norm=1.0)
```

4. Validación Continua

Se monitorea el desempeño en el conjunto de validación tras cada epoch.

python

```
val_loss = val_loss / len(val_loader.dataset)  
val_losses.append(val_loss)
```

Enriquecimiento de Datos

Se incorporan ejemplos de la base de conocimiento para reforzar el entrenamiento.

python

```
# Enriquecer datos de entrenamiento con la base de conocimiento  
entradas_enriquecidas = entradas + self.base_conocimiento  
salidas_enriquecidas = salidas + [8.0] * len(self.base_conocimiento)
```

3. Metodología de Evaluación

El sistema implementa una evaluación multidimensional, más allá de una calificación numérica simple.

Dimensiones Evaluadas

1. Relevancia: Correspondencia con los requisitos.
2. Claridad: Calidad sintáctica, conectores, vocabulario.
3. Profundidad: Inclusión de conceptos clave y análisis.
4. Estructura: Organización, secciones, coherencia.
5. Originalidad: Ideas propias y diversidad léxica.

Cálculo de Relevancia

```
def _calcular_relevancia(self, contenido, titulo, objetivo):  
    """  
        Calcula la relevancia semántica entre el contenido y los  
        requisitos.  
        Utiliza modelos de transformers si están disponibles para un  
        análisis más preciso.  
    """  
    if not titulo and not objetivo:  
        return 1.0  
  
    requisitos = f"{titulo} {objetivo}"  
  
    if self.use_transformer:  
        try:  
            tokens_requisitos =  
self.transformer_tokenizer(requisitos, padding=True,  
truncation=True, return_tensors="pt").to(self.device)  
            tokens_contenido =  
self.transformer_tokenizer(contenido[:512], padding=True,  
truncation=True, return_tensors="pt").to(self.device)  
  
            with torch.no_grad():
```

```

        embedding_requisitos =
self.transformer_model(**tokens_requisitos).last_hidden_state.mean(dim=1)

        embedding_contenido =
self.transformer_model(**tokens_contenido).last_hidden_state.mean(dim=1)

        embedding_requisitos =
torch.nn.functional.normalize(embedding_requisitos, p=2, dim=1)
        embedding_contenido =
torch.nn.functional.normalize(embedding_contenido, p=2, dim=1)

        similarity = torch.matmul(embedding_requisitos,
embedding_contenido.transpose(0, 1)).item()

        return max(0.0, min(1.0, similarity))
    except Exception:
        pass

    try:
        vec_requisitos = self.vectorizer.transform([requisitos])
        vec_contenido = self.vectorizer.transform([contenido])
        similarity = cosine_similarity(vec_requisitos,
vec_contenido)[0][0]
        return max(0.0, min(1.0, similarity))
    except Exception:
        palabras_clave = set(requisitos.split())
        palabras_contenido = set(contenido.split())
        palabras_comunes =
palabras_clave.intersection(palabras_contenido)

        if len(palabras_clave) > 0:
            return len(palabras_comunes) / len(palabras_clave)
        return 0.5

```

Ajuste de Calificación

```

def _ajustar_calificacion(self, calificacion_base, relevancia,
claridad, profundidad, estructura):
    """

```

```
Ajusta la calificación base según relevancia y métricas de
calidad.
"""
    if relevancia < 0.3:
        factor_relevancia = relevancia / 0.3
    else:
        factor_relevancia = 1.0

    factor_calidad = (claridad * 0.3 + profundidad * 0.4 +
estructura * 0.3)
    calificacion_ajustada = calificacion_base * factor_relevancia *
factor_calidad
    return calificacion_ajustada
```

4. Preprocesamiento de Datos

Se aplican técnicas de preprocesamiento textual para preparar los datos antes del análisis.

Limpieza de Texto

python

```
def _preprocess_text(self, text):
    """
    Preprocesa el texto para análisis, preservando más información
    semántica.
    """
    if not text:
        return ""

    text = text.lower()
    text = re.sub(r'^\w\s.,;:¿?¡!()"-'', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

Vectorización TF-IDF

python

```
# Vectorizador TF-IDF mejorado
self.vectorizer = TfidfVectorizer(
    stop_words=self.spanish_stopwords,
    max_features=5000,
    ngram_range=(1, 3),
    min_df=2,
    max_df=0.95
)
```

Dataset Personalizado

python

```
class TextDataset(Dataset):
    def __init__(self, entradas, salidas, tokenizer):
        self.entradas = entradas
        self.salidas = salidas
        self.tokenizer = tokenizer

    def __len__(self):
        return len(self.entradas)

    def __getitem__(self, idx):
        entrada =
self.tokenizer.transform([self.entradas[idx]]).toarray()[0]
        if len(entrada) < 5000:
            entrada = np.pad(entrada, (0, 5000 - len(entrada)),
'constant')
        elif len(entrada) > 5000:
            entrada = entrada[:5000]

        salida = float(self.salidas[idx])
        return torch.tensor(entrada, dtype=torch.float32),
torch.tensor(salida, dtype=torch.float32)
```

5. Generación de Retroalimentación

El sistema genera retroalimentación detallada, personalizada y orientada a la mejora.

Identificación de Fortalezas y Debilidades

python

```
def _identificar_fortalezas_debilidades(self, contenido, metricas,
calificacion):
    """
    Identifica fortalezas y debilidades específicas en el trabajo.
    """
    fortalezas = []
    debilidades = []

    if metricas['claridad'] > 0.7:
        fortalezas.append("Claridad en la exposición de ideas")
    elif metricas['claridad'] < 0.4:
        debilidades.append("Falta de claridad en la exposición de
ideas")

    if metricas['profundidad'] > 0.7:
        fortalezas.append("Análisis profundo del tema")
    elif metricas['profundidad'] < 0.4:
        debilidades.append("Análisis superficial del tema")

    # ... análisis adicional

    return fortalezas, debilidades
```

5. Generación de Retroalimentación

Generación de Comentarios

El sistema genera retroalimentación detallada con base en la calificación obtenida, las fortalezas y debilidades del estudiante, el contenido entregado, y en caso de estar disponible, su estilo de aprendizaje. Este es el esquema básico de funcionamiento:

python

```
def _generar_comentarios_detallados(self, calificacion, contenido,
titulo, objetivo, fortalezas, debilidades, estilo_aprendizaje=None):
    """
    Genera comentarios detallados y personalizados basados en el
    análisis del contenido.
    """
    # Seleccionar plantilla base según calificación
    if calificacion >= 9.0:
        comentario_base =
np.random.choice(self.feedback_templates['excellent'])
    elif calificacion >= 8.0:
        comentario_base =
np.random.choice(self.feedback_templates['very_good'])
    # ... más rangos de calificación

    # Añadir comentarios sobre fortalezas específicas
    comentario = comentario_base
    if fortalezas:
        comentario += "\n\nAspectos destacables:"
        for fortaleza in fortalezas[:3]: # Limitar a 3 fortalezas
            comentario += f"\n- {fortaleza}."

    # ... más personalización

    return comentario
```

Plantillas de Retroalimentación

El sistema utiliza una serie de plantillas clasificadas por desempeño para mantener consistencia y adaptabilidad en los comentarios:

python

```
self.feedback_templates = {
    'excellent': [
        "Excelente trabajo. Has demostrado un dominio completo del
tema, con un análisis profundo y bien estructurado.",
        "Trabajo sobresaliente. Cumples con todos los requisitos y
vas más allá, demostrando comprensión avanzada de los conceptos.",
        # ... más plantillas
    ],
    'very_good': [
        "Muy buen trabajo. Demuestras una sólida comprensión del
tema con un análisis bien desarrollado.",
        # ... más plantillas
    ],
    # ... más categorías
}
```

6. Generación de Prácticas

El sistema puede generar nuevas prácticas educativas a partir de un título y un objetivo de aprendizaje. Utiliza su base de conocimiento para enriquecer la práctica con descripciones, instrucciones y recursos.

Proceso de Generación

python

```
def generar_practica(self, titulo, objetivo):
    """
    Genera una práctica completa y detallada basada en el título y
    objetivo.
    """
    # Combinar título y objetivo
    prompt = f"{titulo}. {objetivo}"

    # Encontrar prácticas similares en la base de conocimiento
    prompt_vec = self.vectorizer.transform([prompt])
    similarities = cosine_similarity(prompt_vec, self.tfidf_matrix)

    # Obtener los 3 ejemplos más similares
    top_indices = similarities[0].argsort()[-3:][::-1]
    practicas_similares = [self.base_conocimiento[i] for i in
top_indices]

    # Determinar la categoría temática
    categoria = self._determinar_categoria_tematica(titulo,
objetivo)

    # Construir la práctica completa
    nueva_practica = {
        "titulo": titulo,
        "objetivo": objetivo,
        "descripcion": descripcion,
        "categoria": categoria,
        "instrucciones": instrucciones,
        "recursos": recursos,
        "criterios_evaluacion": criterios_evaluacion,
```

```
        "objetivos_aprendizaje": objetivos_aprendizaje,
        "tiempo_estimado": "2-3 horas",
        "nivel_dificultad": "Intermedio"
    }

    return nueva_practica
```

Base de Conocimiento

Esta es una muestra del tipo de prácticas almacenadas que sirven como referencia para nuevas generaciones:

python

```
self.base_conocimiento = [
    "Resolver ecuaciones diferenciales de primer orden utilizando métodos numéricos.",
    "Análisis de datos utilizando técnicas estadísticas descriptivas e inferenciales.",
    "Implementación de algoritmos de ordenamiento y búsqueda para estructuras de datos.",
    # ... más ejemplos
]
```

7. Uso del Sistema

Evaluación de Contenido

El sistema puede analizar un contenido entregado por un estudiante y devolver una evaluación integral que incluye calificación, comentarios, métricas y sugerencias.

python

```
def analizar_contenido(self, contenido_archivo, titulo_actividad="",
objetivo_actividad="", estilo_aprendizaje=None):
    """
        Analiza el contenido del archivo y devuelve una calificación y
        comentarios detallados.
        Incorpora análisis semántico profundo, evaluación
        multidimensional y personalización por estilo de aprendizaje.
    """
    self.contador_evaluaciones += 1
    logger.info(f"Iniciando análisis de contenido
#{self.contador_evaluaciones}")

    if not contenido_archivo or not contenido_archivo.strip():
        logger.warning("Contenido vacío detectado")
        return {
            "calificacion": 0.0,
            "comentarios": "El archivo está vacío o no contiene
texto válido.",
            "sugerencias": "Es necesario entregar un trabajo con
contenido para poder evaluarlo.",
            "relevancia": 0.0,
            "recursos_recomendados": []
        }

    # ... procesamiento del análisis

    respuesta = {
        "calificacion": round(calificacion_final, 1),
        "comentarios": comentarios,
        "sugerencias": sugerencias,
        "relevancia": round(relevancia * 100, 1),
        "metricas": {
```

```

        "claridad": round(metricas['claridad'] * 10, 1),
        "profundidad": round(metricas['profundidad'] * 10, 1),
        "estructura": round(metricas['estructura'] * 10, 1),
        "originalidad": round(metricas['originalidad'] * 10, 1)
    },
    "fortalezas": fortalezas[:3],
    "debilidades": debilidades[:3],
    "recursos_recomendados": recursos_recomendados
}

logger.info(f"Análisis de contenido
#{self.contador_evaluaciones} completado")
return respuesta

```

Entrenamiento del Modelo

El modelo se entrena a partir de un conjunto de entregas evaluadas previamente. A continuación, se muestra el proceso estándar:

python

```

# Recopilar datos de entrenamiento
entradas, calificaciones = recopilar_datos_entrenamiento()

# Dividir en entrenamiento y validación
X_train, X_val, y_train, y_val = train_test_split(
    entradas, calificaciones, test_size=0.2, random_state=42
)

# Entrenar el modelo
generador_ml.entrenar_modelo(X_train, y_train, epochs=50,
batch_size=8)

# Evaluar el modelo
evaluacion = evaluar_modelo(X_val, y_val)
print(f"Evaluación del modelo: {evaluacion}")

# Guardar el modelo
torch.save(generador_ml.model.state_dict(), "modelo_practicas.pth")

```

Generación de Prácticas Automática

Una vez entrenado, el modelo puede utilizarse para generar nuevas prácticas:

python

```
# Crear instancia del generador
generador_ml = GeneradorPracticasML()

# Generar una nueva práctica
nueva_practica = generador_ml.generar_practica(
    titulo="Análisis de algoritmos de ordenamiento",
    objetivo="Comparar la eficiencia de diferentes algoritmos de
ordenamiento mediante análisis teórico y empírico"
)

# Guardar la práctica
generador_db.guardar_contenido_generado(practica_id, nueva_practica)
```

8. Consideraciones y Limitaciones

Fortalezas del Sistema

1. **Evaluación Multidimensional:** Considera claridad, profundidad, estructura y originalidad.
2. **Retroalimentación Personalizada:** Comentarios detallados adaptados a cada estudiante.
3. **Adaptabilidad a Estilos de Aprendizaje:** Genera sugerencias según el perfil cognitivo del estudiante.
4. **Robustez ante Errores:** Múltiples validaciones para manejar entradas vacías o no válidas.
5. **Escalabilidad:** Puede ser entrenado con más datos y ajustado a nuevos dominios.

Limitaciones

1. **Dependencia de Datos de Entrenamiento:** La calidad del modelo depende de los ejemplos previos.
2. **Sesgos Lingüísticos:** Optimizado principalmente para textos en español.
3. **Costo Computacional:** Modelos avanzados requieren hardware con capacidad de procesamiento alto.
4. **Baja Interpretabilidad:** Los modelos de aprendizaje profundo no siempre ofrecen trazabilidad clara.
5. **Evaluación de Creatividad:** Dificultad para valorar trabajos altamente creativos o subjetivos.

Recomendaciones de Implementación

- Mantener supervisión humana en los procesos de calificación.
- Actualizar el modelo regularmente con nuevas entregas reales.
- Aplicar validación cruzada para verificar consistencia.
- Recoger feedback de usuarios para afinar retroalimentación.
- Implementación gradual, iniciando con prácticas de bajo riesgo.